

Pay with Visby SDK — Standalone Test Guide

For Judah · 2026-07-09 · Tests the SDK in isolation from the marketplace QA cycle (12b.md). Most steps are automatable — Chrome-Claude prompts are included for each stage.

1. What you are testing (30 seconds)

The SDK lets any merchant site sell a physical item with Visby handling payment + provenance. The chain is: the merchant server calls **POST /api/sdk/checkout** with its secret key → gets a hosted checkout URL → the **<visby-button>** element (public/sdk/v1/button.js) opens that URL in a popup → the buyer pays (card or on-chain) → Visby mints the Tally NFT to the buyer → a signed webhook fires to the merchant → the merchant sees the order/fees on the /merchant dashboard and gets paid their net in USDC by a daily sweep. Fee: 3.5% with a \$0.50 floor.

2. Prerequisites

- A signed-in Visby account (your normal one) at visby.me — the merchant dashboard lives at **/merchant**.
- A webhook catcher: open **webhook.site** and copy your unique URL (free, no account).
- Stripe test card: **4242 4242 4242 4242**, any future expiry, any CVC/ZIP. Declined-card test: **4000 0000 0000 0002**.
- Everything runs on TEST/sandbox keys + devnet today, so no real money moves.
- Test against **https://visby.me** (recommended — the SDK is cross-origin by nature). Local alternative: run `npm run dev` and swap visby.me for localhost:3000 everywhere below.

3. Test flow — 8 checks in order

Check 1 — Create a merchant + get keys. Sign in → /merchant → create a merchant (name: "SDK Test Shop", webhook URL: your webhook.site URL). Copy the **sk_visby_...** secret key when shown (it is shown once; there is a rotate button if lost). PASS: dashboard shows the merchant with key last-4 + webhook URL.

Check 2 — Create a checkout session by API. From any terminal:

```
curl -X POST https://visby.me/api/sdk/checkout \
  -H "Authorization: Bearer sk_visby_YOUR_KEY" \
  -H "Content-Type: application/json" \
  -d '{"product_name":"Test Chronograph","serial_number":"SDKTEST-0001",
    "price":25,"currency":"USD",
    "success_url":"https://example.com/thanks",
    "cancel_url":"https://example.com/cancel"}'
```

PASS: JSON response containing a checkout id/URL (https://visby.me/sdk/checkout/<id>). Use a FRESH serial_number each run — serials are provenance keys and duplicates are rejected.

Check 3 — Merchant page embed. Save this as sdk-test.html and open it in Chrome:

```
<!doctype html><html><body>
  <h1>SDK Test Shop</h1>
  <script src="https://visby.me/sdk/v1/button.js" async></script>
  <visby-button checkout-url="PASTE_CHECKOUT_URL_HERE"></visby-button>
</script>
  document.querySelector('visby-button').addEventListener('visby:complete',
    e => document.title = 'DONE ' + e.detail.order_id + ' ' + e.detail.nft_address);
</script>
```

</body></html>

PASS: the VisbyPay button renders (Quicksand wordmark, gradient) and clicking it opens the hosted checkout popup (480x760).

Check 4 — Complete the purchase. In the popup: sign in (or create an account — NOTE per 12b item P1, an SDK-page signup should eventually auto-create a business account; today it will not, that is a known open item), pay with the 4242 test card. PASS: popup completes, the page title flips to "DONE <order_id> <nft_address>" (the visby:complete event), and success_url redirect works if you let it.

Check 5 — Webhook delivery + re-send. Open your webhook.site tab. PASS: a POST arrived with an HMAC signature header and an order payload (paid/minted). Then on /merchant, find the order and press "Re-send webhook" — a second identical delivery should appear. (Failed deliveries auto-retry via cron with backoff.)

Check 6 — Merchant dashboard numbers. On /merchant: the order shows product, price, status (paid → minted), pay method, webhook badge, and the money split. PASS: platform fee = 3.5% of price (min \$0.50) and merchant net = price – fee. For the \$25 test: fee \$0.88, net \$24.12 (or fee exactly \$0.875 rounded per fee math — verify it matches the order row).

Check 7 — The NFT is real. Take nft_address from Check 4 and look it up on a Solana devnet explorer (explorer.solana.com?cluster=devnet or Helius). PASS: the asset exists and is owned by the buyer wallet. The buyer account also sees it under Profile → My Items.

Check 8 — Failure paths (all by curl, no UI). Each must return a clean JSON error, never a stack trace (ties into 12b ERR1):

```
# bad key -> 401 "Invalid API key"
curl -s -o /dev/null -w "%{http_code}\n" -X POST https://visby.me/api/sdk/checkout \
  -H "Authorization: Bearer sk_visby_WRONG" -H "Content-Type: application/json" -d '{}'
```

```
# missing serial_number -> 400
# price 0 or negative -> 400
# currency EUR -> 400 "Only USD is supported today"
# registry-rejected serial -> 422 serial_rejected:true
# 61 requests in a minute -> 429 rate-limited
# declined card 4000...0002 -> clean decline in popup, NO order, NO mint
```

Bonus — merchant payout sweep. Merchant USDC payouts run as a daily cron (/api/sdk/pay-merchants, 10:00 UTC). I (Claude) can trigger it with CRON_SECRET and confirm the order flips to merchant_payout_status = paid with a tx — ask me to "run the SDK payout sweep check" after a successful purchase.

4. Chrome-Claude prompts (automate most of it)

Paste these one at a time into a Claude session with the Chrome extension connected. Fill the ALL-CAPS placeholders. Each prompt is self-contained.

Prompt A — merchant setup — paste into Claude (Chrome extension connected):

In Chrome, go to <https://webhook.site> and copy my unique URL. Then go to <https://visby.me/merchant> (I am already signed in). Create a merchant named "SDK Test Shop" with that webhook.site URL as the webhook URL. When the secret key (sk_visby_...) is displayed, report it back to me along with the webhook.site URL. Do not navigate away until you have captured the full secret key.

Prompt B — session + test page — paste into Claude (Chrome extension connected):

Create a checkout session: POST <https://visby.me/api/sdk/checkout> with header "Authorization: Bearer SECRET_KEY" and JSON body {"product_name":"Test Chronograph","serial_number":"SDKTEST-RANDOM4DIGITS","price":25,"currency":"USD","success_url":"https://example.com/thanks"}. Then write a local file sdk-test.html containing and a whose checkout-url is the URL you got back, plus a visby:complete listener that sets document.title to "DONE " + order_id + " " + nft_address. Open the file in Chrome and confirm the VisbyPay button renders.

Prompt C — run the checkout — paste into Claude (Chrome extension connected):

On the sdk-test.html page, click the VisbyPay button. In the popup that opens, sign in as me if prompted, choose card payment, and pay with Stripe test card 4242 4242 4242 4242, expiry 12/30, CVC 123, ZIP 78701. Wait for completion, then check the opener page title — it should read "DONE ". Report both values. If anything errors, screenshot it and read the console + network logs for the failing request.

Prompt D — webhook + dashboard verify — paste into Claude (Chrome extension connected):

Open my webhook.site tab and confirm a POST arrived for the order; paste me the payload and the signature header name. Then go to <https://visby.me/merchant>, find the order, and verify: status is minted, platform fee is 3.5% of \$25 (min \$0.50), merchant net = price minus fee, webhook badge shows delivered. Click "Re-send webhook" and confirm a second delivery lands on webhook.site. Report every number you saw.

Prompt E — failure paths — paste into Claude (Chrome extension connected):

Run these against <https://visby.me/api/sdk/checkout> and report each HTTP status + error JSON: (1) Authorization "Bearer sk_visby_WRONG" with any body — expect 401; (2) valid key SECRET_KEY but body missing serial_number — expect 400; (3) valid key, price -5 — expect 400; (4) valid key, currency "EUR" — expect 400. Then on the sdk-test.html popup flow, try card 4000 0000 0000 0002 and confirm a clean decline message appears, no order is created on the merchant dashboard, and nothing was minted. Flag any response that leaks a stack trace or "undefined".

Prompt F — NFT on-chain check — paste into Claude (Chrome extension connected):

Take the nft_address ADDRESS_FROM_PROMPT_C and open <https://explorer.solana.com/address/ADDRESS?cluster=devnet> in Chrome. Confirm the asset exists and report its current owner. Then go to visby.me, open my Profile > My Items, and confirm the Test Chronograph Tally appears there.

5. What only Judah can do

- Approve the Chrome-extension sign-in / 2FA codes when Privy asks (Claude will pause and tell you).
- Nothing else — every SDK check above is browser- or curl-automatable. The marketplace-side gaps the SDK test will expose (SDK signup should auto-create a business account = 12b P1/A7) are already on the fix list; do not fail the SDK for them.

Record results in 12b.md style: each check PASS/FAIL + notes. Send me the list (or the filled interactive checklist) and failures enter the same fix-retest rotation as everything else.